

Ethereum Wallet Checker 학습일지

작성자: 진재화

1. 이번 과제를 통해 처음 알게 된 것 3가지

① HTML, CSS, JavaScript의 역할과 분리 이유

처음 AI가 HTML, CSS, JavaScript를 각각 다른 파일로 작성했을 때 왜 굳이 분리하는지 의문이 들었습니다. 알아보니 HTML은 화면의 구조, CSS는 디자인, JavaScript는 동작을 담당하며, 이를 분리하면 수정하거나 관리하기 편리하다는 것을 알게 되었습니다.

② Ethereum JSON-RPC의 사용 방법

AI가 `eth_getBalance`와 `eth_blockNumber`를 사용한다고 했을 때, 정확히 무엇인지 알아보기 위해 과제에 첨부된 Ethereum 공식 문서를 직접 찾아봤습니다. 각각 지정한 지갑의 잔액과 가장 최근 블록 번호를 반환한다는 것을 알게 되었습니다. 실제로 요청을 보내 16진수 응답도 확인했지만, 아직 내부 동작 원리까지 완전히 이해하지는 못했습니다.

③ 16진수 데이터 변환과 BigInt

RPC 서버가 반환한 잔액은 Wei 단위의 16진수여서 바로 ETH로 표시할 수 없었습니다. AI가 BigInt를 사용한 이유를 확인하면서 일반적인 JavaScript 숫자 처리에서는 큰 정수의 정확도가 떨어질 수 있다는 것을 알게 되었습니다. 이후 1 ETH, 1 Wei, 블록 번호의 변환 결과를 테스트했습니다.

2. AI에게 했던 질문 중 도움이 된 질문 2개

질문 1. "commit을 영어로 해야 해?"

처음에는 커밋 메시지를 한국어로 작성하고 싶었지만, 실무에서 자주 사용하는 영어 커밋 형식을 알게 되었습니다. 이후 `feat: create basic wallet lookup UI`처럼 작업 내용을 구분해 기록했습니다.

질문 2. "Enter 키 기능을 추가한 코드를 지웠는데도 Enter 키로 조회가 되는데?"

AI가 Enter 키로 조회하는 기능을 추가해 줬지만, 바로 커밋하지 않고 정말 필요한 코드인지 확인하고 싶었습니다. 추가된 코드를 지운 상태에서도 Enter 키가 작동하는 것처럼 보여 AI에게 이유를 질문했습니다. 이후 새로고침하고 다시 테스트하니 동작하지 않았고, 코드를 복구하자 정상적으로 작동했습니다. 이 경험을 통해 AI가 작성한 코드를 그대로 받아들이기보다 직접 제거하고 비교하며 검증하는 과정이 중요하다는 것을 느꼈습니다.

3. AI의 제안을 검토한 과정과 문제 해결

AI의 제안을 명확하게 거절하고 다른 방식으로 구현한 사례는 없었습니다. 코드 대부분을 AI가 작성했고, 저는 이를 적용하고 테스트하며 문제가 없는지 확인했습니다.

진행 중 GitHub 원격 저장소 주소를 잘못 설정해 오류가 발생했고, AI의 설명을 참고해 수정했습니다. 로딩 버튼의 CSS 코드를 빠뜨린 사실도 나중에 발견해 추가로 커밋했습니다.

특히 네트워크 오류 테스트에서는 오프라인 상태에서 `abc123`을 입력했지만 예상했던 오류가 나타나지 않았습니다. 이후 잘못된 주소는 RPC 요청 전에 검사된다는 사실을 이해했고, 올바른 주소로 다시 테스트해 네트워크 오류를 확인했습니다. 또한 AI가 제안한 `Promise.all()`을 적용한 뒤 개발자 도구의 Network 탭에서 두 요청이 동시에 실행되는지도 확인했습니다.

4. 아직 이해하지 못한 부분과 앞으로의 학습

이번 프로젝트는 ChatGPT를 전적으로 활용해 구현했기 때문에 아직 코드를 혼자 작성하거나 전체 동작을 설명하기는 어렵습니다. 특히 `async/await`, `Promise.all()`, `BigInt`의 구체적인 동작 방식과 JSON-RPC의 구조는 더 공부해야 합니다.

이번 과제를 진행하면서 AI로 코드를 완성하는 것과 그 코드를 이해하는 것은 별개라는 점을 느꼈습니다. 앞으로는 완성된 코드를 직접 분석하고 일부 기능을 다시 구현해 보면서 부족한 부분을 보완하고 싶습니다.

참고 자료: [Ethereum JSON-RPC 공식 문서](#), ChatGPT(코드 작성·설명·오류 해결·개선 제안)